PROYECTO FINAL

ALUMNA:

Claudia Figueroa

ÍNDICE

PROYECTO FINAL 2026

- Introducción 03
- Objetivo 03
- Situación problemática 04
- Modelo de Negocio 04
- Diagrama Entidad-Relación (E-R) 06
- DER generado a partir del archivo SQL 07
- Listado de tablas 08
- Ingreso de registros 11
- Vistas 11
- Funciones 13
- Procedimientos 14
- Triggers 14

PROYECTO FINAL 2026

INTRODUCCIÓN

El presente proyecto consiste en el diseño de una base de datos para la gestión de un emprendimiento dedicado a la venta de productos para mascotas y artículos de decoración del hogar, como camitas, almohadones en distintos tamaños y productos personalizados mediante estampado DTF. Las ventas se realizan a través de diferentes canales, como Mercado Libre, redes sociales y tienda web, y las entregas pueden efectuarse mediante envío o retiro.
La temática fue elegida por tratarse de un rubro con el que me encuentro familiarizada y que resulta adecuado para modelar procesos reales de ventas, logística y control de stock.
La implementación de una base de datos permite centralizar la información del negocio, organizar los registros y mejorar el seguimiento de las operaciones.
El modelo de negocio corresponde a un esquema de comercio electrónico con producción y personalización propia, en el cual se comercializan productos físicos a través de múltiples canales de venta.

OBJETIVO

El objetivo del proyecto es diseñar y desarrollar una base de datos relacional que permita gestionar de manera estructurada la información clave del emprendimiento, contemplando las distintas áreas que intervienen en su funcionamiento.
Desde el punto de vista operativo, la base de datos permitirá administrar el catálogo de productos, sus variantes por tamaño o tipo, el registro de clientes y la gestión de pedidos.
En el aspecto logístico, facilitará el seguimiento de las modalidades de entrega, ya sea mediante envíos o retiros, así como el control del stock a través de movimientos de inventario.
En el área contable, el sistema permitirá registrar pagos asociados a cada pedido, los importes y estados de las transacciones, lo que contribuye al control de ingresos y a la organización financiera del emprendimiento.
Por último, desde una perspectiva analítica, la base de datos permitirá obtener información relevante para la toma de decisiones, como productos más vendidos, canales de venta con mayor volumen de pedidos o evolución de las ventas en el tiempo.

SITUACIÓN PROBLEMÁTICA

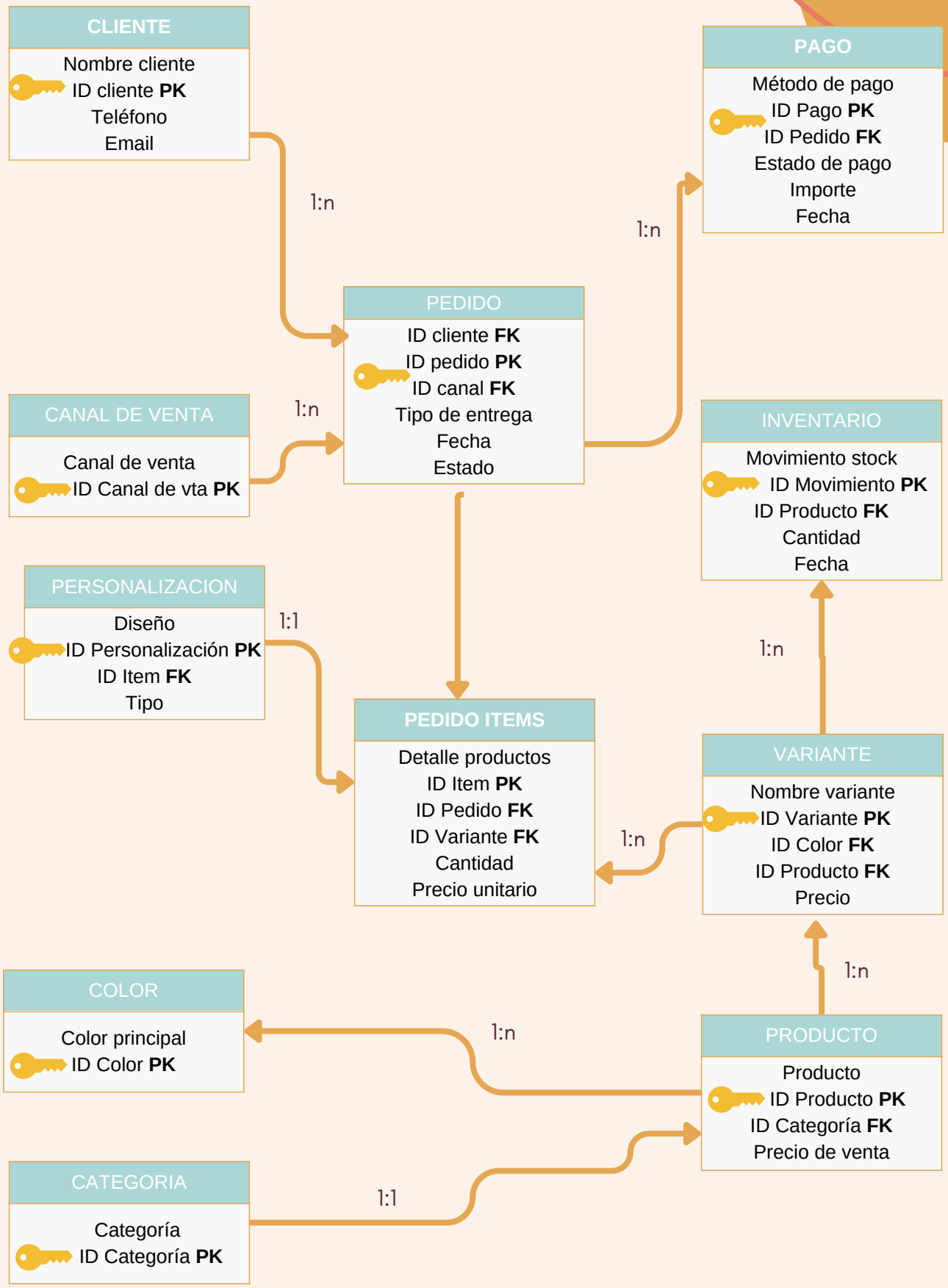
En un emprendimiento que comercializa productos a través de distintos canales, la información de pedidos, clientes, pagos y stock suele manejarse de manera dispersa, lo que dificulta el control y el seguimiento de las operaciones.
La falta de un sistema organizado puede generar errores en el registro de ventas, pérdida de control del stock y dificultades para gestionar envíos o retiros. Además, sin datos estructurados resulta complejo analizar la información y obtener indicadores útiles para la toma de decisiones.
La implementación de una base de datos permite centralizar y relacionar la información del negocio, mejorar el control operativo y contar con datos confiables para la gestión y el análisis.

MODELO DE NEGOCIO

El emprendimiento corresponde a un modelo de comercio electrónico de productos físicos, en el cual se diseñan, producen o personalizan artículos para mascotas y decoración del hogar, que luego se comercializan a través de distintos canales digitales, principalmente Mercado Libre, redes sociales y tienda web.
El proceso del negocio comienza con la publicación y administración del catálogo de productos, que pueden presentarse en diferentes variantes, como tamaños o tipos. A partir de allí se reciben pedidos de clientes, se preparan los productos —incluyendo en algunos casos tareas de personalización o estampado— y finalmente se coordina la entrega, que puede realizarse mediante envío o retiro.
El funcionamiento del emprendimiento involucra distintas áreas que requieren gestión de información: el control de productos y stock, el registro de pedidos y clientes, la administración de pagos y el seguimiento de las entregas. La base de datos propuesta permite integrar estos procesos en un único sistema, facilitando la organización y el control de las operaciones.

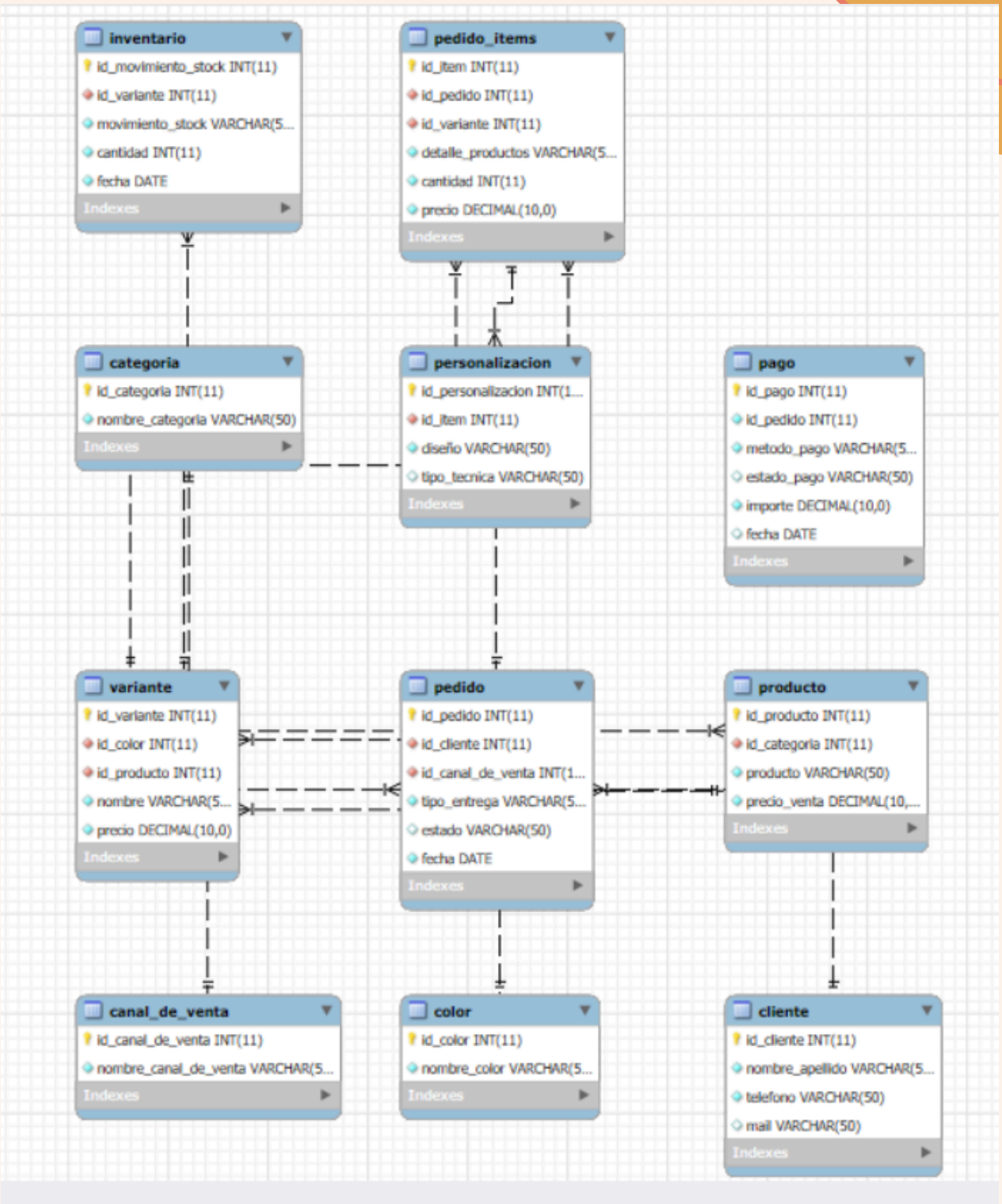
El diagrama representa el flujo general del negocio, desde la administración del catálogo hasta la entrega del pedido, incluyendo los procesos de registro, preparación y control de pagos.

DIAGRAMA ENTIDAD-RELACIÓN (E-R)



El diagrama Entidad-Relación representa la estructura lógica de la base de datos del emprendimiento, mostrando las entidades principales, sus atributos y las relaciones entre ellas. El modelo coincide con el esquema generado a partir del script SQL y refleja los procesos de gestión de productos, pedidos, clientes, pagos y entregas.

DER GENERADO A PARTIR DEL ARCHIVO SQL



El diagrama entidad-relación representa la estructura lógica de la base de datos diseñada para la gestión de un emprendimiento de venta de productos para mascotas y artículos de decoración. En el modelo se identifican las principales entidades del sistema, tales como clientes, pedidos, productos, variantes, pagos e inventario, así como las relaciones existentes entre ellas.

Cada pedido se asocia a un cliente y a un canal de venta, y puede contener uno o más ítems, los cuales corresponden a variantes específicas de productos. A su vez, cada variante pertenece a un producto y puede presentar diferentes características como color, precio y stock, que se controlan mediante los movimientos de inventario.

El modelo permite también registrar pagos y personalizaciones asociadas a los ítems de cada pedido.

LISTADO DE TABLAS

ENTIDAD: CLIENTE

CAMPO	TIPO DE DATO	CLAVE
ID CLIENTE	INT AUTO INCREMENTAL	PK
NOMBRE Y APELLIDO	VARCHAR(50)	-
TELÉFONO	VARCHAR(50)	-
MAIL	VARCHAR(50)	-

ENTIDAD: PEDIDO

CAMPO	TIPO DE DATO	CLAVE
ID CLIENTE	INT	FK
ID PEDIDO	INT AUTO INCREMENTAL	PK
ID CANAL	INT	FK
TIPO DE ENTREGA	VARCHAR(50)	-
FECHA	DATE	-
ESTADO	VARCHAR(50)	-

ENTIDAD: PAGO

CAMPO	TIPO DE DATO	CLAVE
ID PAGO	INT AUTO INCREMENTAL	PK
MÉTODO DE PAGO	VARCHAR(50)	-
ID PEDIDO	INT	FK
ESTADO DE PAGO	VARCHAR(50)	-
IMPORTE	DECIMAL	-
FECHA	DATE	-

ENTIDAD: CANAL DE VENTA

CAMPO	TIPO DE DATO	CLAVE
ID CANAL DE VENTA	INT AUTO INCREMENTAL	PK
CANAL DE VENTA	VARCHAR(50)	-

ENTIDAD: PERSONALIZACIÓN

CAMPO	TIPO DE DATO	CLAVE
ID ITEM	INT	FK
ID PERSONALIZACIÓN	INT AUTO INCREMENTAL	PK
DISEÑO	VARCHAR(50)	-
TIPO DE TÉCNICA	VARCHAR(50)	-

ENTIDAD: COLOR

CAMPO	TIPO DE DATO	CLAVE
ID COLOR	INT AUTO INCREMENTAL	PK
COLOR	VARCHAR(50)	-

ENTIDAD: ITEMS DEL PEDIDO

CAMPO	TIPO DE DATO	CLAVE
ID ITEM	INT AUTO INCREMENTAL	PK
DETALLE PRODUCTOS	VARCHAR(50)	-
ID PEDIDO	INT	FK
ID VARIANTE	INT	FK
CANTIDAD	INT	-
PRECIO POR UNIDAD	DECIMAL	-

ENTIDAD: CATEGORÍA

CAMPO	TIPO DE DATO	CLAVE
ID CATEGORÍA	INT AUTO INCREMENTAL	PK
CATEGORÍA	VARCHAR(50)	-

ENTIDAD: PRODUCTO

CAMPO	TIPO DE DATO	CLAVE
ID PRODUCTO	INT AUTO INCREMENTAL	PK
ID CATEGORIA	INT	FK
PRODUCTO	VARCHAR(50)	-
PRECIO DE VENTA	DECIMAL	-

ENTIDAD: VARIANTE

CAMPO	TIPO DE DATO	CLAVE
ID VARIANTE	INT AUTO INCREMENTAL	PK
NOMBRE	VARCHAR(50)	-
ID COLOR	INT	FK
ID PRODUCTO	INT	FK
PRECIO	DECIMAL	-

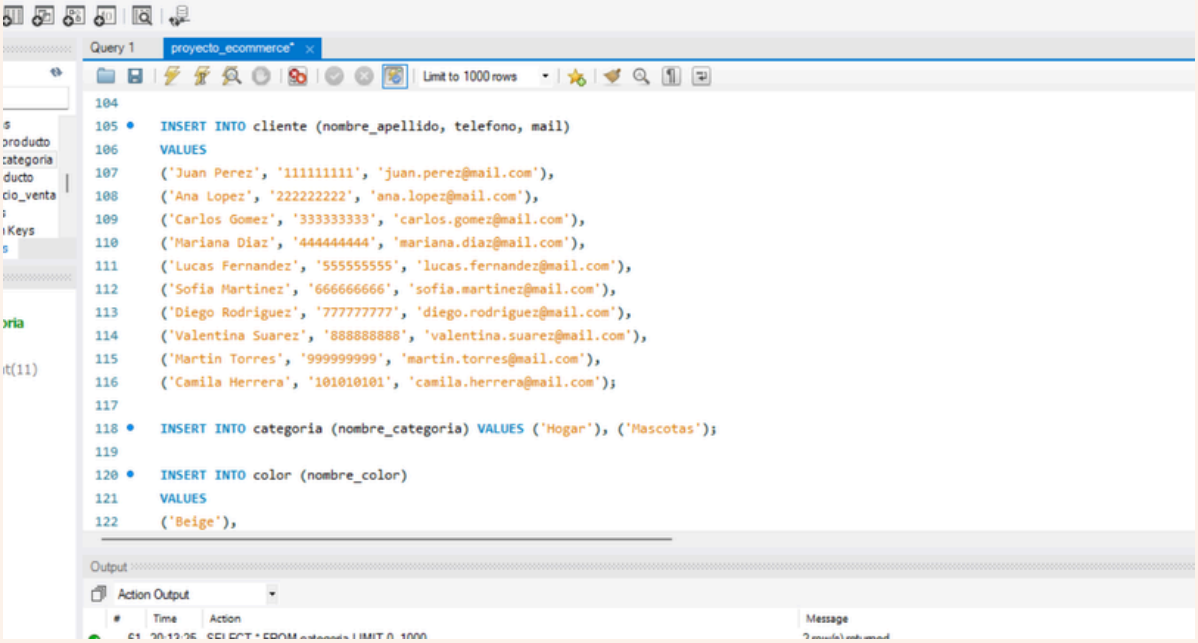
ENTIDAD: INVENTARIO

CAMPO	TIPO DE DATO	CLAVE
ID MOV STOCK	INT AUTO INCREMENTAL	PK
MOVIMIENTO DE STOCK	VARCHAR(50)	-
ID PRODUCTO	INT	FK
FECHA	DATE	-
CANTIDAD	INT	-

INGRESO DE REGISTROS

Para ingresar los registros a las tablas, se utilizó el comando INSERT INTO. El procedimiento se realizó siguiendo estos pasos:

1. Se seleccionó la tabla y se definieron las columnas específicas a completar.
2. En lugar de realizar una sentencia por cada fila, se utilizó una única instrucción INSERT INTO seguida de una lista de valores separados por comas.
3. Asignación de Valores. Se respetó el orden de las columnas declaradas.
4. Luego de correr cada sentencia, se validó los datos mediante una consulta SELECT *, confirmando que los tipos de datos ingresados coincidieran con las restricciones de cada tabla y que todas estaban completas.



```
INSERT INTO cliente (nombre_apellido, telefono, mail)
VALUES
('Juan Perez', '111111111', 'juan.perez@mail.com'),
('Ana Lopez', '222222222', 'ana.lopez@mail.com'),
('Carlos Gomez', '333333333', 'carlos.gomez@mail.com'),
('Mariana Diaz', '444444444', 'mariana.diaz@mail.com'),
('Lucas Fernandez', '555555555', 'lucas.fernandez@mail.com'),
('Sofia Martinez', '666666666', 'sofia.martinez@mail.com'),
('Diego Rodriguez', '777777777', 'diego.rodriguez@mail.com'),
('Valentina Suarez', '888888888', 'valentina.suarez@mail.com'),
('Martin Torres', '999999999', 'martin.torres@mail.com'),
('Camila Herrera', '101010101', 'camila.herrera@mail.com');

INSERT INTO categoria (nombre_categoria) VALUES ('Hogar'), ('Mascotas');

INSERT INTO color (nombre_color)
VALUES
('Beige'),
```

VISTAS

VISTA_TOTAL_PEDIDOS

El objetivo es calcular el importe total de cada pedido según los productos incluidos. Permite obtener el total de cada pedido sumando la multiplicación de la cantidad por el precio de cada ítem. Es útil para análisis de ventas. Las tablas utilizadas son las de producto y la de producto_item.

```
CREATE OR REPLACE VIEW vista_total_pedidos AS
SELECT
    p.id_pedido,
    SUM(pi.cantidad * pi.precio) AS total_pedido
FROM pedido p
JOIN pedido_items pi ON p.id_pedido = pi.id_pedido
GROUP BY p.id_pedido
;
SELECT * FROM vista_total_pedidos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	total_pedido
1	55000
2	30000
3	60000
4	45000
5	15000
6	25000
7	52000
8	90000

VISTA_TOTAL_PEDIDOS

El objetivo es identificar los pedidos con envío a domicilio. Esta vista filtra los pedidos según el tipo de entrega, mostrando únicamente aquellos que deben ser enviados. Las tablas involucradas son la de pedidos y la de cliente.

```sql
CREATE OR REPLACE VIEW vista_pedidos_envio AS
SELECT
    p.id_pedido,
    c.nombre_apellido AS cliente,
    p.tipo_entrega
FROM pedido p
JOIN cliente c ON p.id_cliente = c.id_cliente
WHERE p.tipo_entrega = 'Envio';

SELECT * FROM vista_pedidos_envio;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	tipo_entrega
1	Juan Perez	Envio
3	Carlos Gomez	Envio
4	Mariana Diaz	Envio
6	Sofia Martinez	Envio
7	Diego Rodriguez	Envio
9	Martin Torres	Envio
10	Camila Herrera	Envio

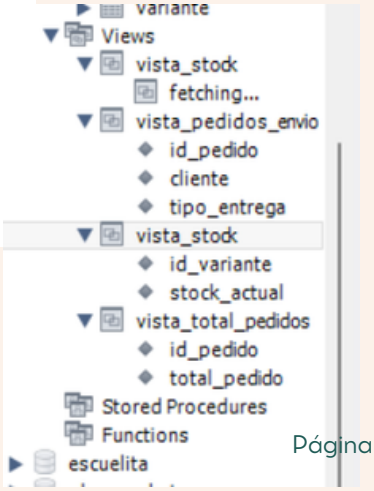
VISTA_STOCK

El objetivo es determinar el stock disponible de cada variante de producto. La vista calcula el stock actual a partir de los movimientos registrados en la tabla inventario, sumando ingresos y restando salidas. Permite conocer la disponibilidad de productos en tiempo real. Utiliza la tabla inventario.

```sql
CREATE OR REPLACE VIEW vista_stock AS
SELECT
    id_variante,
    SUM(cantidad) AS stock_actual
FROM inventario
GROUP BY id_variante;
SELECT * FROM vista_stock;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_variante	stock_actual
1	18
2	20
3	20
4	15
5	10
12	-1
20	-3
23	-1
24	-1



FUNCIONES PERSONALIZADAS

FN_TOTAL_CLIENTE

El objetivo de esta función es calcular cuanto gastó cada cliente en su pedido. Utiliza la tabla de pedidos y la tabla cliente. Los datos que utiliza son el precio de cada producto comprado y la cantidad. Luego hice un SELECT para que salga el nombre del cliente con un AS.

```
SELECT
 c.nombre_apellido,
 fn_total_cliente(c.id_cliente) AS total_gastado
FROM cliente c
WHERE c.id_cliente = 1;
```

```
-- FUNCION 1

DELIMITER //
CREATE FUNCTION fn_total_cliente(p_id_cliente INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
DECLARE total DECIMAL(10,2);
SELECT SUM(pi.cantidad * pi.precio)
INTO total
FROM pedido p
JOIN pedido_items pi ON p.id_pedido = pi.id_pedido
WHERE p.id_cliente = p_id_cliente;
RETURN IFNULL(total, 0);
END //

DELIMITER ;
SELECT fn total cliente (1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre_apellido	total_gastado

FN_CANTIDAD_POR_CLIENTE

Esta función permite obtener la cantidad de pedidos realizados por un cliente a partir de su identificador. Cuando hice la carga de los registros, cargué un solo pedido por cliente. Asi que la función siempre me daba 1. Agregué, más registros en pedido con el mismo ID de cliente, para probar que la función estuviese bien.

```
-- AGREGUÉ PEDIDOS CON EL MISMO ID DE CLIENTE
INSERT INTO pedido (id_cliente, id_canal_de_venta, tipo_entrega, estado, fecha)
VALUES
(1, 1, 'Envio', 'Pendiente', '2026-03-10'),
(5, 2, 'Retiro', 'Entregado', '2026-02-28'),
(3, 3, 'Envio', 'Pendiente', '2026-02-15'),
(9, 1, 'Envio', 'Entregado', '2026-03-20'),
(5, 2, 'Retiro', 'Pendiente', '2026-02-25'),
(5, 3, 'Envio', 'Entregado', '2026-03-10'),
(2, 1, 'Envio', 'Pendiente', '2026-03-22'),
(2, 2, 'Retiro', 'Entregado', '2026-02-15'),
(2, 3, 'Envio', 'Pendiente', '2026-02-16'),
(2, 1, 'Envio', 'Entregado', '2026-03-25')
;
```

```
-- FUNCION 2

DELIMITER //
CREATE FUNCTION fn_cantidad_por_cliente(p_id_cliente INT)
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE cantidad INT;
SELECT count(*)
INTO cantidad
FROM pedido
WHERE id_cliente = p_id_cliente;
RETURN IFNULL(cantidad, 0);
END //
DELIMITER ;

SELECT fn_cantidad_por_cliente(9);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fn_cantidad_por_cliente(9)
2

PROCEDIMIENTOS

CREAR_PEDIDO

Este procedimiento almacenado permite poder ingresar los pedidos de forma estructurada y reutilizable sin estar haciendo un INSERT.

```
DELIMITER //

CREATE PROCEDURE crear_pedido
(
    IN p_id_cliente INT,
    IN p_id_pedido int,
    IN p_tipo_entrega VARCHAR (50),
    in p_fecha DATE
)
BEGIN
INSERT INTO pedido (id_cliente, id_canal, tipo_entrega, fecha)
VALUES (p_id_cliente, p_id_canal, p_tipo_entrega, p_fecha);
END //

DELIMITER ;
SHOW PROCEDURE STATUS WHERE Db = 'ecommerce';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
ecommerce	crear_pedido	PROCEDURE	root@localhost	2026-03-27 21:35:35	2026-03-27 21:35:35	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_gen

ACTUALIZAR_ESTADO

Este procedimiento almacenado permite cambiar el estado de un pedido. Los procedimientos se llaman con CALL nombre_procedimiento (parametros). Como ejemplo se cargaron los registros del procedimiento crear_pedido.

```
-- PROCEDIMIENTO 2

DELIMITER //

CREATE PROCEDURE actualizar_estado(
    IN p_id_pedido INT,
    IN p_estado VARCHAR(50)
)
BEGIN
    UPDATE pedido
    SET estado = p_estado
    WHERE id_pedido = p_id_pedido;
END //

DELIMITER ;
SHOW PROCEDURE STATUS WHERE Db = 'ecommerce';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Data Colla
ecommerce	actualizar_estado	PROCEDURE	root@localhost	2026-03-27 22:30:22	2026-03-27 22:30:22	DEFINER		utf8mb4	utf8mb4_general_ci	utf8m
ecommerce	crear_pedido	PROCEDURE	root@localhost	2026-03-27 22:03:54	2026-03-27 22:03:54	DEFINER		utf8mb4	utf8mb4_general_ci	utf8m

TRIGGERS

DESCONTAR_STOCK

Este Trigger permite descontar stock automáticamente al vender. Utiliza la tabla de pedidos_item y la de inventario. Cuando ingresa un registro en pedido_item, se descuenta de la tabla inventario. Evita olvidar actualizar stock y automatiza el sistema.

```
DELIMITER //

CREATE TRIGGER descontar_stock
AFTER INSERT ON pedido_items
FOR EACH ROW
BEGIN
    INSERT INTO inventario (id_variante, movimiento_stock, cantidad, fecha)
    VALUES (NEW.id_variante, 'Salida', -NEW.cantidad, CURDATE());
END //

DELIMITER ;
```

ESTADO_PENDIENTE

Asignar estado "Pendiente" automáticamente. Cuando ingresa un pedido en la tabla pedido, el estado queda vacío y automáticamente completa con pendiente.

```
DELIMITER //

CREATE TRIGGER estado_pendiente
BEFORE INSERT ON pedido
FOR EACH ROW
BEGIN
    IF NEW.estado IS NULL THEN
        SET NEW.estado = 'Pendiente';
    END IF;
END //

DELIMITER ;
```